

```
Terminal
block_cipher = None

a = Analysis(
    ['frame.py'],
    pathex=['/home/codio/workspace/day'],
    binaries=[],
    datas=[],
    hiddenimports=[],
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
    excludes=[],
    win_no_prefer_redirects=False,
    win_private_assemblies=False,
    cipher=block_cipher,
    noarchive=False,
)
pyz = PyZ(a.pure, a.zipped_data, cipher=block_cipher)

exe = EXE(
    pyz,
    a.scripts,
    [],
    exclude_binaries=True,
    name='day',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    console=True,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)

coll = COLLECT(
    exe,
    a.binaries,
    a.zipfiles,
    a.datas,
    strip=False,
    upx=True,
    upx_exclude=[],
    name='day',
)

(day) codio@morningorlando-misterjoseph:~/workspace/day$ conda deactivate
(base) codio@morningorlando-misterjoseph:~/workspace/day$
```

Guide

Collapse

One-Folder Approach

Bundling the Executable

Start the `day` virtual environment.

conda activate day

Now that we have a working Python program, it is time to bundle it up as an executable. We are going to use the default method, which generates an executable and a folder of information. The `frame.py` script is the entry point for our program, so we will run Pyinstaller on this file. By default, this will create an executable called `frame`. That is not a good description of what this program does. We should change the name of the final program. In addition, we need to specify where Pyinstaller can find the `date.py` script so we can import the `get_info` function.

In the terminal, run the following command. Pyinstaller will bundle `frame.py` into an executable. The `--name` flag allows us to rename the program to `day`. Finally, the `--paths` flag tells Pyinstaller where to look for the `date.py` file. Give the full path to the file.

pyinstaller frame.py --name day --paths="/home/codio/workspace/day"

Pyinstaller will prints lots of text to the terminal. We are looking for three pieces of information to let us know the process finished. List all of the contents of the `day` directory. In addition to our Python files, we should see a `day.spec` file as well as two directories called `build` and `dist`.

ls

The `build` directory contains the files created during the build process. This information can help debug the program should Pyinstaller now work as expected. The `dist` directory contains another directory called `day`. This directory contains bundled program and all of the required files. List its contents. You should see all kinds of files, including the `day` executable (colored green).

ls dist/day

In Linux, use `./` before the name of an executable program. This is similar to double-clicking on an icon on a Mac or Windows machine. Since the `day` executable is in the `day` directory which is in the `dist` directory, the command would be:

./dist/day/day

Try these variations:

Run the program so that it outputs information about yesterday.

./dist/day/day yesterday

Run the program so that it outputs information about tomorrow.

./dist/day/day tomorrow

Exploring the Pyinstaller Output

People wanting to run your program need the executable file *and* all of the information in the folder. In our case, that means the `day` directory found inside the `dist` directory. You cannot just share the executable file `day` all by itself. It will not run. Users need the entire directory. If you want to share your work, you could zip up the folder to share with others.

We tend to think of an executable file as being a solitary file that can exist on its own. By default, Pyinstaller does not do this. Instead it creates a single folder with all that you need to run the program. While the one-folder approach makes sharing a bit cumbersome, it does allow you greater flexibility when debugging your work.

If everything works as intended, you will not need to interact with the `build` directory. However, there is one file in there that may be of use. In our example, it is called `warn-day.txt`. This text file contains any warnings from the build and bundle process. A common problem with Pyinstaller are modules that it cannot find. If you try to run your executable and see an import error, consult the warn file for more information.

Lastly, the spec file is an important file and deserves a bit of an explanation. The spec file tells Pyinstaller how to build and bundle your scripts. Running Pyinstaller creates a generic spec file. However, in our case, a generic file would not work. We needed to make a few changes, which are done through the `--name` and `--path` flags. Enter the following command to list the contents of the `day.spec` file.

cat day.spec

In the `Analysis` section, you will see the path we added to help Pyinstaller find the `date.py` file.

pathex=['/home/codio/workspace/today'],

In the `EXE` section, you will see the new name to be used with the executable file.

name='day',

The spec file is a key component to creating an executable bundle with Pyinstaller. You can add data files, other binary files, set options for specific platforms, etc. See the documentation for more information.

conda deactivate

Reading Question

Which command runs the Pyinstaller on the Python script `main.py`?

☒ pyinstaller main.py

☐ python --pyinstaller main.py

☐ pyinstaller --create main.py

☐ pyinstaller my_script.py

The correct answer is:

pyinstaller main.py

You do not use the `python` command or the `--create` flag to generate an executable. Be sure to pass Pyinstaller `main.py` and not `my_script.py`.

Check It

Next